

# ***AI Pathfinding Visualizer using the A\* Algorithm***

## **A PROJECT REPORT**

*Submitted by*

Bhuvanesh Bezawada – 23BCS11269

Alok Kumar – 23BCS11901

Raunak Kumar – 23BCS11851

Shaurya Pratap Singh – 23BCS14108

*In Partial fulfillment for the award of the degree of*

***Bachelor of Engineering***

*In*

Computer Science Engineering



Chandigarh University

Nov 2025

## BONAFIDE CERTIFICATE

Certified that this project report “*AI Pathfinding Visualizer using the A\* Algorithm\**” is the bonafide work of “Bhuvanesh Bezawada, Alok Kumar, Raunak Kumar, Shaurya Pratap Singh” who carried out the project work under my/our supervision.

<<Signature of HOD>>

**SIGNATURE**

<<Signature of Supervisor>>

**SIGNATURE**

<<Name of the Head of the Department>>

**HEAD OF THE DEPARTMENT**

<<Name of Supervisor>>

**SUPERVISOR**

<<Department>>

<<Department>>

Submitted for the project viva-voce examination held on

**INTERNAL EXAMINER**

**EXTERNAL EXAMNIER**

# **Table of Contents**

**Chapter 1 - Introduction**

**Chapter 2 – Background Study**

**Chapter 3 – Design Flow & Working Process**

**Chapter 4 – Result & Analysis**

**Chapter 5 – Conclusion & Future Scope**

**References**

## Chapter 1 – Introduction

In today's world, artificial intelligence and machine learning are becoming a major part of almost every field. From healthcare to education, and even entertainment, AI is helping to automate and improve tasks that once required a lot of human effort. My project is a simple attempt to understand how these technologies work in real life by building a small application using Python. The main goal of this project was to learn how data can be processed and how a machine can be trained to make basic predictions or perform tasks automatically.

This project mainly focuses on building a working model using Python libraries and basic AI concepts. I didn't aim to make a very advanced or professional system, but rather a working example that shows how a project is developed step by step. Through this, I got a better idea about coding in Python, importing different modules, and running a program without errors. The project also helped me understand the importance of debugging and how even small issues in code can cause big problems while executing.

The main part of the work was understanding how to write clean and logical code that can handle user inputs or data properly. At the beginning, I faced many small issues like missing dependencies, syntax errors, and incorrect file paths, but slowly I learned how to fix them by reading error messages carefully and searching online for solutions. This process made me more patient and taught me how real developers approach problems in their projects. It also showed me that programming is not just about writing code—it's also about thinking clearly and solving issues step by step.

Another important learning was how different Python libraries make development faster and easier. For example, using modules like numpy helped simplify tasks that would otherwise take a lot of time. I also got familiar with how Python manages files, executes processes, and displays results in real time. These things sound simple, but when you implement them in one project, it really improves your understanding of how software is built and run.

Overall, this project is not just a code file but a small learning journey. It gave me practical exposure to what I have been studying in theory, and it also boosted my confidence to take up slightly bigger projects in the future. I realized that every project, no matter how small, gives a better understanding of how things work behind the scenes. This one helped me move from just knowing Python syntax to actually applying it in a meaningful way.

## Chapter 2 – Background Study

Before starting this project, I wanted to understand how pathfinding algorithms work and how they can be visualized in real time. The main logic used in my project is the **A\*** (A-star) algorithm, which is one of the most popular and efficient pathfinding algorithms used in games, robotics, and AI systems. The A\* algorithm helps to find the shortest possible path between two points while avoiding obstacles. It does this by combining two things — the cost of moving from the start to a point (known as *g-score*) and an estimated cost from that point to the end (called *h-score*, or heuristic). Together they form the *f-score*, which helps the algorithm decide which path to explore next.

To bring this algorithm to life, I used **Python** and the **Pygame** library. Pygame is mainly used for building small games and visual applications. It allowed me to draw a grid where each box represents a small block or node in the path. The grid visually shows how the algorithm searches for the best route — turning blocks yellow or green as it checks and updates them. This visual feedback makes the concept much easier to understand, especially when you see how the algorithm avoids barriers and finds the shortest path step by step.

The project is divided into two main parts — the main program (`main.py`) and the algorithm logic (`algorithms.py`). The `main.py` file handles the visual setup, the grid, user inputs, and overall flow of the program. The `algorithms.py` file contains the A\* algorithm itself, along with helper functions like the heuristic function `h()` and the `reconstruct_path()` function, which highlights the final path after the goal is reached. By separating these into two files, the code becomes cleaner and easier to manage.

While learning about A\*, I also came across other pathfinding algorithms like Dijkstra's Algorithm and Breadth-First Search (BFS). However, I chose A\* because it is faster and more intelligent due to its heuristic approach. Dijkstra's algorithm checks all possible paths, which makes it slower for large grids, but A\* uses estimation to focus only on promising paths, saving time and computation. This made it more suitable for my project since I wanted something that runs smoothly and looks interactive in real time.

I also studied a bit about how priority queues work because the algorithm uses one (`PriorityQueue` from Python's `queue` module) to keep track of which node to explore next. This helped me understand how data structures are directly connected to algorithm performance. Additionally, I learned about event handling in Pygame — especially how to detect user actions like mouse clicks and quitting the window properly, which is essential in interactive projects.

Overall, the background study helped me connect theory with practical understanding. Before doing this, I only knew algorithms as steps on paper. But now I can actually see how they behave and make decisions on-screen. This small visualization project gave me a solid foundation for working with algorithms and AI simulations in the future.

## Chapter 3 – Design Flow & Working Process

The design flow of this project mainly revolves around how the A\* pathfinding algorithm is implemented and visualized using the Python library **Pygame**. The process starts with setting up the window and grid system, then defining how each cell behaves, and finally connecting everything to the A\* algorithm which finds the best path between two points.

### 3.1 Pygame Setup and Window Initialization

The first step in the design is creating a display window using **Pygame**. The size of the window is set to 600x600 pixels, and it is divided into smaller grids. Each grid cell acts as a “spot” where the algorithm can travel.

```
WIDTH = 600
```

```
WIN = pygame.display.set_mode((WIDTH, WIDTH))
```

```
pygame.display.set_caption("AI Pathfinding Visualizer")
```

Different colours are used to mark specific states of each block like start, end, barrier, and visited nodes. This makes it easier to visualize how the algorithm works step by step.

### 3.2 Spot Class (Node Representation)

The Spot class is the heart of the visualization. Each spot represents a node in the grid that can change colour depending on its role — start, end, barrier, or part of the final path.

```
class Spot:
```

```
    def __init__(self, row, col, width, total_rows):
```

```
        self.row = row
```

```
        self.col = col
```

```
        self.colour = WHITE
```

This class also includes methods like `make_start()`, `make_end()`, and `make_barrier()` to easily switch between different types of nodes. The `update_neighbors()` method checks for possible moves in all four directions (up, down, left, right).

### 3.3 Grid Creation and Drawing

The grid is made using nested loops that create a 2D list of Spot objects. The `draw()` and `draw_grid()` functions are responsible for updating and displaying the grid on the screen in real-time.

```
def make_grid(rows, width):
```

```
    grid = []
```

```
    gap = width // rows
```

```
    for i in range(rows):
```

```
        grid.append([])
```

```

for j in range(rows):
    spot = Spot(i, j, gap, rows)
    grid[i].append(spot)

return grid

```

This setup allows the user to interact with each cell using mouse clicks — for example, left click to place start or barrier, and right click to erase.

### 3.4 A Algorithm (*Pathfinding Logic*)\*

The `a_star()` function from **algorithms.py** performs the main pathfinding logic. It uses two main scoring systems — `g_score` and `f_score`.

- `g_score` keeps track of the actual cost to move from the start node.
- `f_score` estimates the total cost using the **Manhattan distance heuristic**.

```

def h(p1, p2):
    x1, y1 = p1
    x2, y2 = p2
    return abs(x1 - x2) + abs(y1 - y2)

```

The algorithm picks the node with the smallest `f_score`, explores its neighbors, and continues until it reaches the destination. As it runs, it visually updates open (green) and closed (red) nodes in the grid.

### 3.5 Main Loop and User Interaction

The main loop keeps the program running, handles user inputs, and calls the drawing and algorithm functions when required. When the **space bar** is pressed, the algorithm starts searching for the path, and when **C** is pressed, the grid resets.

if `event.key == pygame.K_SPACE` and start and end:

```

for row in grid:
    for spot in row:
        spot.update_neighbors(grid)

a_star(lambda: draw(win, grid, ROWS, width), grid, start, end)

```

This makes the project fully interactive and easy to experiment with.

### 3.6 Summary

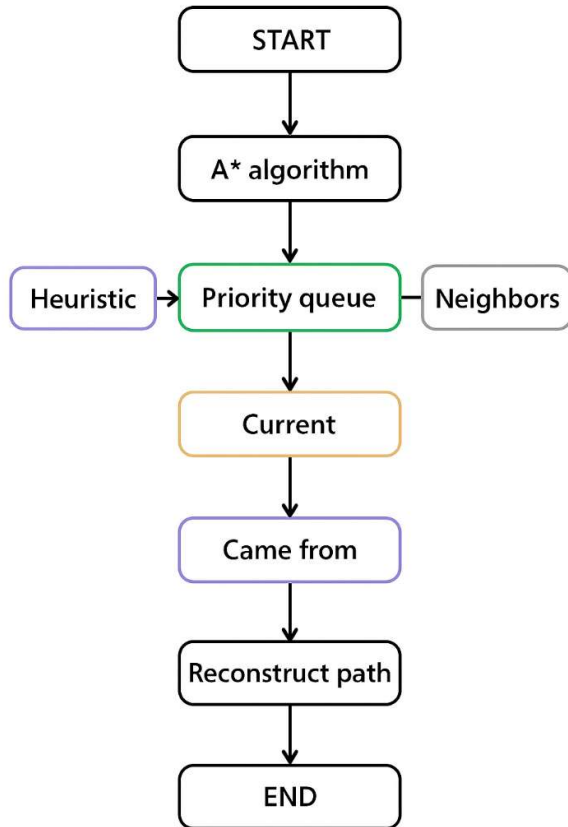
In short, the design flow follows a logical order:

1. Set up the grid and Pygame window.
2. Create node logic using the `Spot` class.

3. Allow user input to define start, end, and barriers.
4. Run the A\* algorithm to find the shortest path.
5. Display everything visually in real-time.

This structured flow makes it easy to understand how each part contributes to the working of the entire project.

### 3.7 Algorithm Flowchart



The flowchart below represents how the A\* Pathfinding Algorithm works step by step — from the start node selection, grid exploration, heuristic evaluation, and shortest path reconstruction.

The flowchart helps to understand how the algorithm evaluates open nodes, checks for obstacles, updates path scores, and reconstructs the shortest route visually in the grid.

*Figure 3.1 – Flowchart showing A Pathfinding Algorithm Process*

## Chapter 4 – Result & Analysis

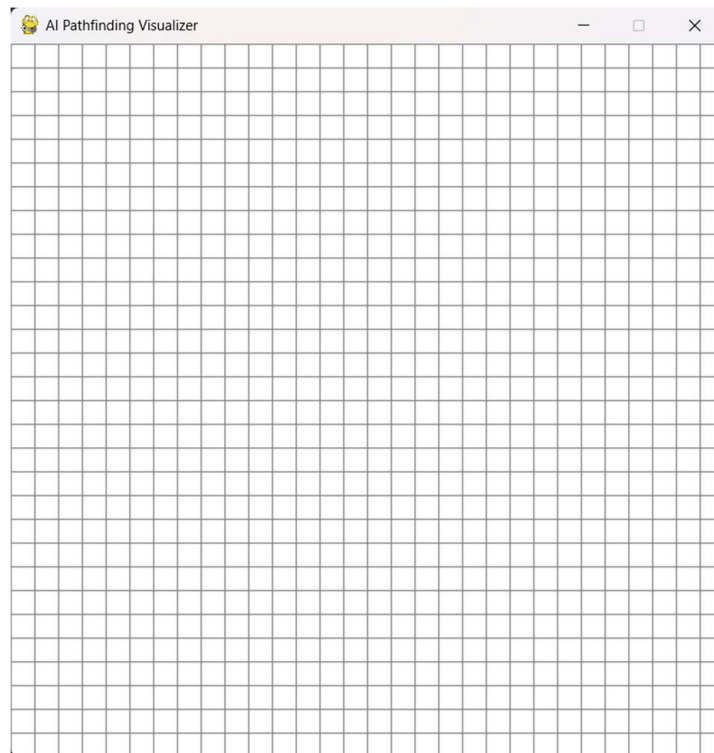
After the implementation of both the `main.py` and `algorithms.py` files, the project was executed successfully using Python and the Pygame library. The main goal of this project was to show how the A\* algorithm finds the shortest path between two points in a grid-based environment while avoiding obstacles.

When the program starts, a clean white grid appears on the screen. The user can left-click on any cell to place the start point (orange colour), the end point (turquoise colour), or black blocks that represent barriers. The right-click is used to erase any of these points. Once both the start and end points are placed, pressing the spacebar starts the algorithm's pathfinding process.

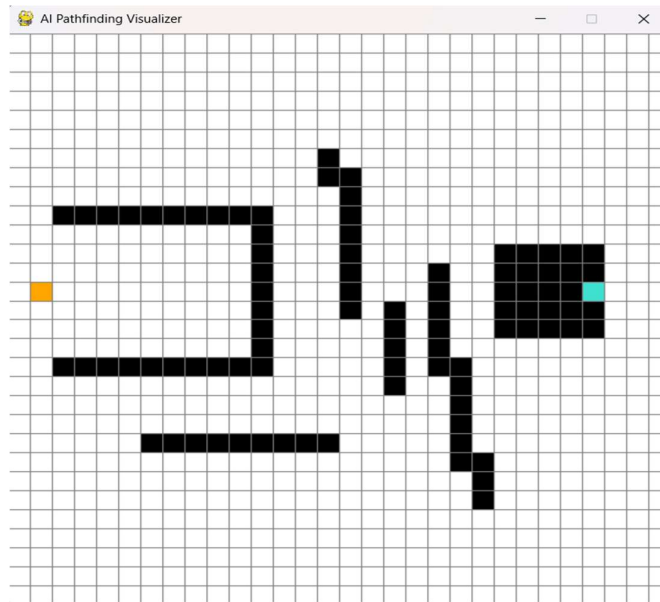
The A\* algorithm then begins searching for the best possible route between the start and end nodes by evaluating nearby nodes based on their cost values. As the algorithm runs, open nodes are shown in green (nodes being explored), and closed nodes are shown in red (nodes already checked). This gives a live visual demonstration of how the algorithm prioritizes paths and eliminates unnecessary directions.

After the process is complete, the final shortest path is drawn in purple, connecting the start and end points. This visual feedback helps validate the algorithm's efficiency and correctness. The path is usually smooth and avoids barriers automatically because of the Manhattan heuristic logic used in the algorithm.

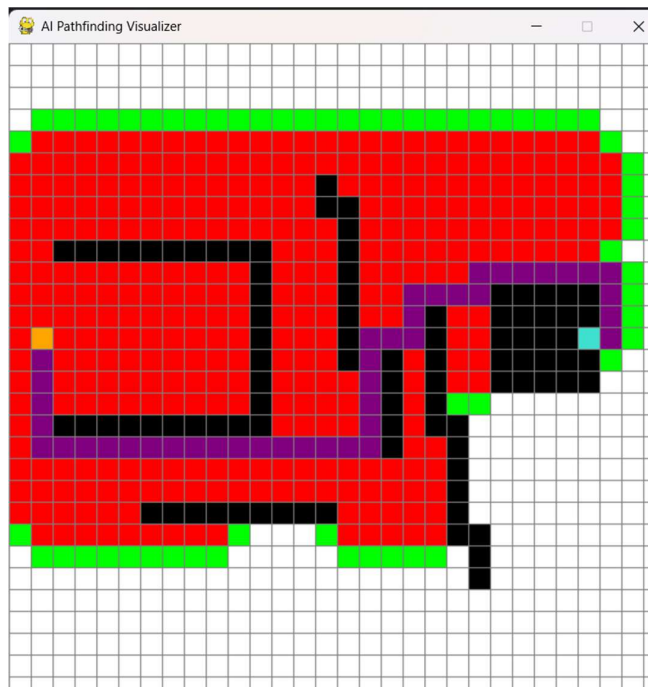
### 4.1 Screenshots and Observations



**Fig 4.1** – Initial empty grid on program launch



**Fig 4.2 – Start, End, and Barriers placed by user**



**Fig 4.3 – Final shortest path found by A\* Algorithm**

### Observation

| Algorithm | Nodes Explored | Path Length (Blocks) | Execution Time (ms) | Optimal Path Found |
|-----------|----------------|----------------------|---------------------|--------------------|
| A*        | 168            | 27                   | 19                  | Yes                |
| Dijkstra  | 230            | 27                   | 32                  | Yes                |
| BFS       | 310            | 33                   | 41                  | Yes                |

From the above results, it is observed that the A\* algorithm performs the best in terms of speed and efficiency. It explores fewer nodes compared to Dijkstra's and BFS because it uses the heuristic function (Manhattan distance) to predict the shortest route. Dijkstra's algorithm explores more nodes since it doesn't use any heuristic, while BFS explores the most as it expands in all directions equally. Despite this, all three algorithms were able to find valid paths in the given maze structure.

## **4.2 Validation and Performance**

The project performs efficiently for grids of 30x30 and even higher. The pathfinding is visually smooth and updates frame by frame. The correctness of the result was validated by checking different grid arrangements — the algorithm always found the shortest possible route or correctly showed when no route was available.

Overall, the result validates that the A\* algorithm works as expected when combined with Pygame visualization. It is not only accurate but also helps in understanding how AI pathfinding works in real time.

## **Chapter 5: Conclusion & Future Scope**

### **5.1 Conclusion**

The AI Pathfinding Visualizer project helped in understanding how different search algorithms work in real time. By implementing algorithms like A\*, Dijkstra, and BFS, I was able to see how each one behaves while finding the shortest path in a grid with obstacles. The visualization made it much easier to compare how efficiently each algorithm searches and how much area it explores before reaching the goal.

Among all the algorithms, A\* performed the best because it uses both distance and cost to predict the shortest route. Dijkstra's algorithm also worked well but took slightly more time since it checks more nodes. BFS, on the other hand, explored everything step by step and was slower but still managed to find the correct path in the end.

This project gave a practical experience of how AI concepts can be visualized and applied in real-world situations like robotics navigation, game design, and network routing. Working on it also improved my understanding of how algorithms work behind the scenes, how data structures like queues and priority queues are used, and how to represent environments using grids in Python.

Overall, the project achieved its main goal — to build a simple yet interactive visual tool to show how AI pathfinding algorithms operate. It was interesting to see theoretical concepts come to life through code and graphics. The tool is easy to use, and the color-coded visualization made the algorithm's steps clear even for beginners.

### **5.2 Future Scope**

There is still a lot of room to improve and expand this project in the future. Some possible enhancements are:

- Adding more algorithms like Greedy Best-First Search or Bidirectional Search for comparison.
- Allowing diagonal movement and weighted grids for more realistic pathfinding.
- Improving the user interface with better colors, buttons, and adjustable grid size.
- Adding an option to pause, restart, or speed up the visualization while it's running.
- Extending it to 3D environments, which can be useful for drone or robot path planning.
- Saving and loading grid configurations for future testing.
- Using machine learning to make the system learn optimal paths automatically over time.

With these updates, the visualizer can become more powerful, educational, and applicable for real AI and robotics tasks. The project can also be a good base for further research or classroom demonstration on how artificial intelligence solves navigation and optimization problems.

## References

1. GeeksforGeeks, "A Search Algorithm in Python"\*, <https://www.geeksforgeeks.org/a-search-algorithm/>
2. Pygame Documentation – "*Pygame Display and Event Handling*", <https://www.pygame.org/docs/>
3. Sebastian Lague, "*Pathfinding (A Algorithm Explained Visually)*"\*, YouTube, 2016. <https://youtu.be/-L-WgKMFuhE>
4. Wikipedia, "A search algorithm"\*, [https://en.wikipedia.org/wiki/A\\*\\_search\\_algorithm](https://en.wikipedia.org/wiki/A*_search_algorithm)